

Uso de LLMS em Inspeções na Engenharia de Software

Fábio A. Bocampagni¹, Giovanni F. Ornellas², Hugo A. Souza¹, Gabriel M. Rebello³

¹Instituto de Computação – Universidade Federal do Rio de Janeiro (UFRJ)
Rio de Janeiro – RJ – Brasil

Abstract. *In recent years, Large Language Models (LLMs) have demonstrated significant potential in various areas of information technology, including software engineering and cybersecurity. Trained on vast datasets of textual data, these models can comprehend and generate programming code, offering new tools to enhance software quality and security. This article explores the use of LLMs in code inspections, which are essential to ensure software quality and security before deployment. Through a review of previous studies, we highlight how LLMs can be applied to detect vulnerabilities, improve code readability, and identify bugs more efficiently, thereby enhancing software development practices and raising standards of quality in the industry.*

Resumo. *Nos últimos anos, os Modelos de Linguagem Grande (LLMs) têm demonstrado potencial significativo em várias áreas da tecnologia da informação, incluindo engenharia de software e cibersegurança. Estes modelos, treinados em vastos conjuntos de dados textuais, podem compreender e gerar código de programação, proporcionando novas ferramentas para melhorar a qualidade e a segurança do software. Este artigo explora o uso de LLMs em inspeções de código, essenciais para garantir a qualidade e segurança do software antes de sua implantação. Através da revisão de estudos anteriores, destacamos como os LLMs podem ser aplicados para detectar vulnerabilidades, melhorar a legibilidade do código e identificar bugs com maior eficiência, potencializando as práticas de desenvolvimento de software e elevando os padrões de qualidade na indústria.*

1. Introdução

Nos últimos anos, os Modelos de Linguagem Grande (LLMs) têm demonstrado um potencial significativo em diversas áreas da tecnologia da informação, incluindo engenharia de software, cibersegurança e muitas outras. Estes modelos, que são treinados em vastos conjuntos de dados textuais, possuem a capacidade de compreender e gerar código de programação, fornecendo assim novas ferramentas para melhorar a qualidade e a segurança do software.

Este artigo pretende explorar o uso de LLMs especificamente em inspeções de código, uma prática essencial para garantir a qualidade e segurança do software antes de sua implantação. Através da revisão e síntese das contribuições de estudos anteriores, destacamos como os LLMs podem ser aplicados para detectar vulnerabilidades, melhorar a legibilidade do código e identificar bugs com maior eficiência, potencializando as práticas de desenvolvimento de software e elevando os padrões de qualidade na indústria.

2. Referencial Teórico

2.1. Artigo 1 - Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach

Autores: Haonan Li, Yu Hao, Yizhuo Zhai, e Zhiyun Qian

Introdução: Este artigo explora a integração de Modelos de Linguagem Grande (LLMs) com análise estática para melhorar a detecção de bugs em grandes bases de código, como o kernel do Linux. A análise estática é essencial para identificar bugs, mas enfrenta desafios de precisão e escalabilidade em códigos complexos e extensos.

Metodologia: A metodologia proposta, denominada LLift, combina análise estática com a capacidade de compreensão de código dos LLMs, visando superar as limitações tradicionais da análise estática e oferecer uma solução mais eficaz para a detecção de bugs. O foco inicial é a detecção de bugs de Uso Antes da Inicialização (UBI) no kernel do Linux.

Resultados e Avaliação: O LLift apresentou uma taxa de precisão razoável (50%) e conseguiu identificar quatro bugs previamente desconhecidos no kernel do Linux, validados pela comunidade, demonstrando a eficácia da integração de LLMs com análise estática.

Discussão e Trabalhos Futuros: Sugere-se a expansão da integração de LLMs para outros tipos de bugs, como uso após liberação e análise de fluxo de dados, prometendo uma análise mais aprofundada e precisa de casos complexos.

Conclusão: A combinação de análise estática com LLMs tem potencial para aprimorar a detecção de bugs em grandes bases de código, melhorando a precisão e escalabilidade, além de abrir novas direções para a pesquisa na detecção de vulnerabilidades de software.

2.2. Artigo 2 - Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities

Autores: Avishree Khare, Saikat Dutta, Ziyang Li, Alaia Solko-Breslin, Rajeev Alur, Mayur Naik

Introdução: O artigo investiga a capacidade dos LLMs, como GPT-4 e CodeLlama, em detectar vulnerabilidades de segurança em software. O estudo avalia a eficácia dos LLMs em termos de desempenho, explicabilidade e robustez, usando benchmarks diversificados de segurança.

Metodologia: A pesquisa analisa modelos fechados (GPT-4) e abertos (CodeLlama) utilizando benchmarks padronizados e códigos reais, garantindo determinismo nos resultados com temperatura zero para evitar variações nos outputs dos modelos.

Resultados e Avaliação: A pesquisa aponta que LLMs, embora não superem ferramentas de análise estática de última geração, possuem potencial significativo e áreas promissoras de aplicação em segurança de software.

2.3. Artigo 3 - Introdução ao Uso de Inteligência Artificial na Detecção de Vulnerabilidades de Segurança em Projetos de Software

Autores: Zoltán Szabó e Vilmos Bilicki

Introdução: O artigo apresenta uma visão geral do uso de inteligência artificial (IA) para a detecção de vulnerabilidades de segurança em projetos de software, destacando o papel dos LLMs.

Metodologia: A metodologia abrange a aplicação de técnicas de IA, incluindo LLMs, para identificar vulnerabilidades em código-fonte, utilizando análises estáticas e dinâmicas.

Resultados e Avaliação: Estudos de caso mostram que a aplicação de LLMs aumenta a precisão e a eficiência na detecção de vulnerabilidades em diferentes contextos de desenvolvimento de software.

Discussão e Trabalhos Futuros: A pesquisa sugere explorar novas técnicas de IA e ampliar o uso de LLMs em diferentes tipos de vulnerabilidades e linguagens de programação.

Conclusão: O uso de LLMs em detecção de vulnerabilidades de segurança é promissor, oferecendo melhorias substanciais na precisão e eficiência, e abrindo caminho para futuras pesquisas e aplicações.

2.4. Artigo 4 - Using Large Language Models to Better Detect and Handle Software Vulnerabilities and Cyber Security Threats

Autores: Farid Feyzi

Introdução: Este artigo explora como os LLMs podem ser utilizados para melhorar a detecção e o manejo de vulnerabilidades de software e ameaças cibernéticas.

Metodologia: A pesquisa implementa LLMs em sistemas de segurança para detectar e mitigar ameaças, utilizando análises avançadas de dados e estratégias de resposta automática.

Resultados e Avaliação: A aplicação de LLMs demonstrou melhorias significativas na detecção precoce de vulnerabilidades e na resposta a incidentes de segurança, reduzindo o tempo de reação e mitigação.

Discussão e Trabalhos Futuros: Futuras pesquisas podem focar na personalização de LLMs para diferentes ambientes de segurança e na integração com outras tecnologias emergentes.

Conclusão: LLMs são ferramentas poderosas para aprimorar a segurança cibernética, proporcionando uma detecção mais rápida e eficiente de ameaças e vulnerabilidades.

2.5. Artigo 5 - How Readable is Model-generated Code? Examining Readability and Visual Inspection of GitHub Copilot

Autores: Naser Al Madi

Introdução: O artigo avalia a legibilidade e a inspeção visual do código gerado por modelos, com foco no GitHub Copilot.

Metodologia: A pesquisa analisa a legibilidade do código gerado por Copilot através de testes de usabilidade e estudos com desenvolvedores para avaliar a clareza e a facilidade de entendimento do código.

Resultados e Avaliação: Os resultados indicam que o código gerado por Copilot é geralmente legível, mas apresenta variações na qualidade que podem afetar a compreensão e a manutenção do código.

Discussão e Trabalhos Futuros: A pesquisa sugere melhorias nos modelos para aumentar a consistência da legibilidade do código e explorar formas de integrar feedback de desenvolvedores nos modelos.

Conclusão: Embora o GitHub Copilot gere código legível em muitos casos, há espaço para melhorias na consistência e na qualidade do código gerado, o que pode beneficiar significativamente os desenvolvedores.

3. Relato de Experiência

3.1. Busca pelos artigos

Não foi fácil achar bons artigos na área de inspeção que estivessem dentro do tema das LLMs. Muitos artigos abordavam somente o uso de LLMs para confecção/análise de código, mas deixavam de lado a parte de inspeção do mesmo, e vice versa. Além disso, não nos pareceu que existam um grande número de artigos sobre o tema. Como exemplo, em um dos artigos encontrados o autor falava sobre o uso do ChatGPT 2.0 para análise estática, o que acabou arrancando algumas risadas destes autores. Em parte por conta dessa dificuldade, a maior parte dos artigos utilizados se refere a área de cybersegurança, o que faz muito sentido quando se pensa sobre isso. Afinal, muitas vezes grandes falhas na segurança do código conseguem ser notadas durante a inspeção do mesmo, e é preferencial que se faça assim, dado que esse tipo de problema tende a custar muito dinheiro e credibilidade as empresas. Portanto, faz sentido que ajam mais artigos dentro desse tema e que use as LLMs para melhorar a eficácia das análises.

3.2. Experiência própria

Para realizar os testes com as diferentes ferramentas decidimos utilizar um programa básico mas que tivesse espaço para erro. Para tanto, modelamos o fluxograma de um programa simples, mas que possui diversos conceitos importantes na computação, como tipagem, iterações, checagens, I/O e outros. Apesar de não ter um objetivo concreto óbvio o programa conta com uma saída esperada, que servirá como o critério básico para a avaliação das ferramentas a serem testadas. A versão que será apresentada para os modelos foi propositalmente falha e ineficiente, com redundâncias e implementações incorretas da modelagem.

3.3. Especificação do Código

A especificação do código é baseada na seguinte modelagem:

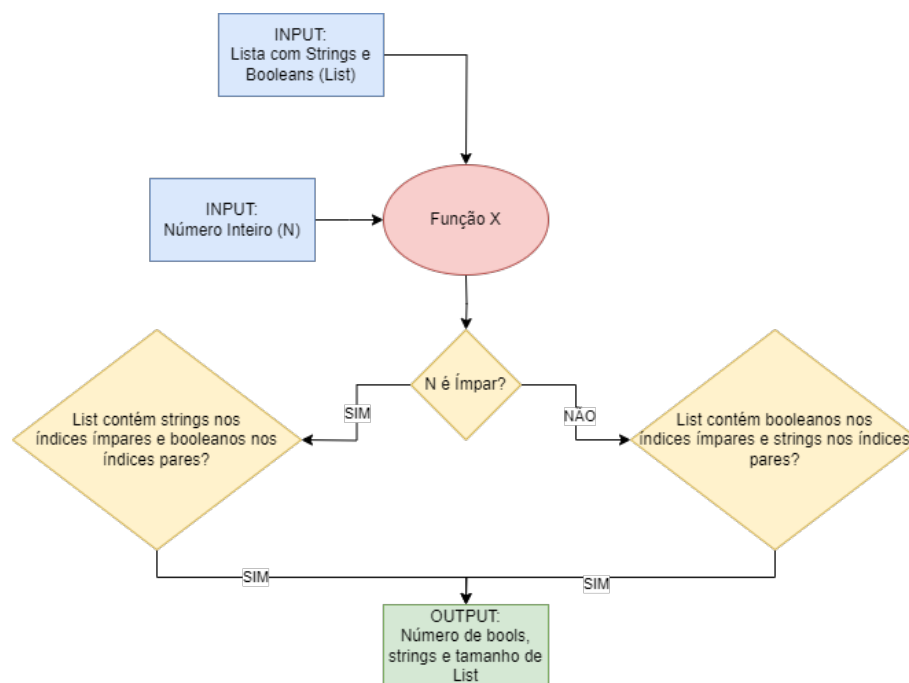


Figure 1. Modelagem da Especificação do Código

- Verificar se o número de entrada é ímpar ou par.
- Se o número for ímpar, verificar se a lista contém strings nos índices ímpares e booleanos nos índices pares.
- Se o número for par, verificar se a lista contém booleanos nos índices ímpares e strings nos índices pares.
- Tratar o índice 0 como o primeiro ímpar já que é a "primeira posição" da lista.
- Se a verificação for bem-sucedida, exibir o tamanho da lista, o número de strings e o número de booleanos.

O código fornecido possui várias falhas e ineficiências que esperamos que sejam detalhadas a seguir pela ferramenta testada. Segue o código original submetido para revisão:

```

def check_odd_even(number, input_list):
    if number % 2 == 0:
        is_odd = False
    else:
        is_odd = True

    strings_count = 0
    booleans_count = 0

    for i in range(len(input_list)):
        if (i + 1) % 2 == 0:
            if is_odd:
                if isinstance(input_list[i], str):
                    strings_count += 1
                elif isinstance(input_list[i], bool):
                    booleans_count += 1
                else:
                    print(f"Invalid element type at index {i}: {input_l
            else:
                if isinstance(input_list[i], bool):
                    booleans_count += 1
                elif isinstance(input_list[i], str):
                    strings_count += 1
                else:
                    print(f"Invalid element type at index {i}: {input_l
        else:
            if is_odd:
                if isinstance(input_list[i], bool):
                    booleans_count += 1
                elif isinstance(input_list[i], str):
                    strings_count += 1
                else:
                    print(f"Invalid element type at index {i}: {input_l
            else:
                if isinstance(input_list[i], str):
                    strings_count += 1
                elif isinstance(input_list[i], bool):
                    booleans_count += 1
                else:
                    print(f"Invalid element type at index {i}: {input_l

    print(f"List size: {len(input_list)}")
    print(f"Number of strings: {strings_count}")
    print(f"Number of booleans: {booleans_count}")

```

```

number = 3

```

```
input_list = [True, "hello", False, "world", True, "python"]
check_odd_even(number, input_list)
```

Segue o prompt passado para a ferramenta

Tenho um script em Python que deve seguir as seguintes especificações, quando um número input for ímpar, o código deve checar se a lista input contém booleans nas posições ímpares e strings nas posições pares, quando o número for ímpar, o código deve checar se a lista contém strings nas posições ímpares e booleans nas posições pares. Se a checagem der certo, o código deve ter como saída o tamanho da lista e o número de booleans e strings. Apesar de a lista começar no índice 0, esse deve ser tratado como o primeiro número ímpar, já que é a "primeira posição" da lista. Faça uma revisão do código a seguir, analisando se ele está de acordo com essas especificações, além disso procure ineficiências no código, sempre explicitando o que considera errado e como consertar o problema. Segue o código: *código base apresentado acima*

3.4. ChatGPT-3.5

A ferramenta que decidimos testar foi o ChatGPT, especificamente a versão gratuita, 3.5, um modelo bastante famoso, mas também conhecido por cometer muitos erros em diversas tarefas, incluindo programar. A interface já muito familiar a todos facilita o uso da ferramenta, e a formatação das respostas permite copiar o código revisado com apenas um clique.

3.4.1. Revisão do Código Fornecido

Erro na Verificação de Índices: A especificação indica que o índice 0 deve ser tratado como ímpar. O código usa incorretamente $(i + 1) \% 2 == 0$ e $(i + 1) \% 2 != 0$ para determinar posições pares e ímpares, respectivamente. Esta lógica falha ao tratar o índice 0 como ímpar. **Correção:** Use $i \% 2 == 0$ para posições ímpares e $i \% 2 != 0$ para posições pares.

Checagem de Tipos Redundante: A lógica de checagem de tipos é repetida tanto nas verificações de índices ímpares quanto pares. **Correção:** Consolide a lógica de checagem de tipos para evitar redundância.

Loop Ineficiente: O loop processa a lista inteira mesmo quando um tipo de elemento inválido é encontrado. As instruções de impressão são usadas para indicar erros, mas não terminam o loop. **Correção:** Introduza tratamento de erro para sair do loop ou da função quando um tipo de elemento inválido for detectado.

Contagem Incorreta de Tipos: As contagens de strings e booleanos não são separadas corretamente com base na paridade do número. **Correção:** Assegure-se de que as contagens de tipos sejam incrementadas com base no índice e na paridade do número.

Lógica Ineficiente: A lógica para lidar com diferentes casos (número ímpar/par e índice ímpar/par) é excessivamente complicada e repetitiva. **Correção:** Simplifique a lógica lidando com os diferentes casos de maneira mais direta.

Código Corrigido e Otimizado

```
def check_odd_even(number, input_list):
    is_odd = number % 2 != 0

    strings_count = 0
    booleans_count = 0

    for i in range(len(input_list)):
        if (is_odd and i % 2 == 0) or (not is_odd and i % 2 != 0):
            if isinstance(input_list[i], str):
                strings_count += 1
            else:
                print(f"Error: Invalid type at odd index {i}: {input_list[i]}")
                return
        else:
            if isinstance(input_list[i], bool):
                booleans_count += 1
            else:
                print(f"Error: Invalid type at even index {i}: {input_list[i]}")
                return

    print(f"List size: {len(input_list)}")
    print(f"Number of strings: {strings_count}")
    print(f"Number of booleans: {booleans_count}")

number = 3
input_list = [True, "hello", False, "world", True, "python"]
check_odd_even(number, input_list)
```

Explicação Detalhada das Correções Verificação de Índices: Alterada a lógica de verificação dos índices para lidar corretamente com a especificação onde o índice 0 é tratado como ímpar. (`is_odd` e `i % 2 == 0`) garante que, se o número for ímpar, elementos nos índices 0, 2, 4, etc. são verificados como strings. (`not is_odd` e `i % 2 != 0`) garante que, se o número for par, elementos nos índices 1, 3, 5, etc. são verificados como strings.

Consolidação da Checagem de Tipos: As condições foram combinadas para evitar redundância na checagem de tipos.

Tratamento de Erros: Introduzidas instruções de retorno para sair da função imediatamente ao encontrar um tipo de elemento inválido.

Contagem Correta de Tipos: As contagens de strings e booleanos foram corretamente incrementadas com base na nova lógica de índices.

Lógica Simplificada: A lógica geral foi simplificada ao mesclar condições semelhantes e reduzir código repetitivo.

Esta versão corrigida e otimizada agora deve realizar as verificações necessárias com precisão e fornecer a saída esperada com base nas especificações originais.

3.5. ChatGPT-4o

A segunda ferramenta foi um segundo modelo, também da OpenAi, O ChatGPT-4. Esse modelo possui diversas melhorias se comparado a versão 3.5, como uma melhor compreensão do contexto devido a uma maior utilização de tokens e um dicionário de embeddings maior. Além disso, o modelo foi treinado com uma database mais recente e é capaz de buscar informações na web.

3.5.1. Código Corrigido

```
def check_odd_even(number, input_list):
    is_odd = number % 2 != 0

    strings_count = 0
    booleans_count = 0

    for i, element in enumerate(input_list):
        if (is_odd and i % 2 == 0) or (not is_odd and i % 2 != 0):
            if isinstance(element, str):
                strings_count += 1
            else:
                print(f"Error: Invalid type at odd index {i}: {element}")
                return
        else:
            if isinstance(element, bool):
                booleans_count += 1
            else:
                print(f"Error: Invalid type at even index {i}: {element}")
                return

    print(f"List size: {len(input_list)}")
    print(f"Number of strings: {strings_count}")
    print(f"Number of booleans: {booleans_count}")

number = 3
input_list = [True, "hello", False, "world", True, "python"]
check_odd_even(number, input_list)
```

Além das correções já feitas no modelo 3.5, a versão 4 trouxe algumas mudanças pro código referente a legibilidade que possivelmente podem ajudar na hora de dar manutenção no código. Dentre elas encontra-se:

Iteração com enumerate

O uso de enumerate no loop for é mais eficiente e legível do que utilizando range() e len(). Ele permite obter o índice e o elemento ao mesmo tempo.

Cheque de Tipo

O código checa se um elemento é uma string ou booleano, e imprime uma mensagem de erro e retorna se encontrar um tipo inválido. Esse comportamento está correto e não há ineficiências significativas aqui.

Clareza e Legibilidade:

A legibilidade do código foi melhorada ao usar enumerate, tornando o código mais direto e fácil de entender.

3.6. Gemini

A terceira alternativa foi o modelo do Google, o Gemini. Em comparação ao ChatGpt3, o Gemini possui acesso a informação atualizada através de pesquisa do google e garante resposta com base em informações recentes. Possui uma API para desenvolvedores e foi treinado com um grande volume de dados em português o que pode ser um grande fator de ajuda quando há recorrência de utilização desse modelo.

Em uma primeira análise o Gemini é muito mais expressivo em suas correções do que os modelos da OpenAi, mostrando claramente e explicando de forma verbosa onde pontos que foram endereçados e como foram endereçados. O código não foi reduzido porém foi corrigido e erros foram tratados. O que mostra um caráter muito mais conservador comparado aos modelos da OpenAi.

3.6.1. Código Corrigido

```
def check_odd_even(number, input_list):
    if not isinstance(number, int):
        raise TypeError("O valor de 'number' deve ser um número inteiro")

    is_odd = number % 2 != 0
    strings_count = 0
    booleans_count = 0
    invalid_element_found = False

    for i in range(len(input_list)):
        if (i + 1) % 2 == 0:
            if is_odd:
                if isinstance(input_list[i], str):
                    strings_count += 1
                elif isinstance(input_list[i], bool):
                    booleans_count += 1
            else:
                invalid_element_found = True
```

```

        break
    else:
        if isinstance(input_list[i], bool):
            booleans_count += 1
        elif isinstance(input_list[i], str):
            strings_count += 1
        else:
            invalid_element_found = True
            break
    else:
        if is_odd:
            if isinstance(input_list[i], bool):
                booleans_count += 1
            elif isinstance(input_list[i], str):
                strings_count += 1
            else:
                invalid_element_found = True
                break
        else:
            if isinstance(input_list[i], str):
                strings_count += 1
            elif isinstance(input_list[i], bool):
                booleans_count += 1
            else:
                invalid_element_found = True
                break

if invalid_element_found:
    strings_count = 0
    booleans_count = 0

print(f"Tamanho da lista: {len(input_list)}")
print(f"Número de strings: {strings_count}")
print(f"Número de booleans: {booleans_count}")

number = 3
input_list = [True, "hello", False, "world", True, "python"]
check_odd_even(number, input_list)

```

Considerações sobre a correção O código original apresentava algumas ineficiências e falhas que o impediam de funcionar conforme as especificações. As principais alterações realizadas foram:

Simplificação da Estrutura

A estrutura aninhada complexa foi substituída por dois loops aninhados, um para elementos pares e outro para ímpares, tornando o código mais legível e intuitivo.

Eliminação de Redundâncias

A verificação repetitiva de `is_odd` foi eliminada armazenando o resultado em uma variável, otimizando o código.

Tratamento Aprimorado de Elementos Inválidos

Um flag `invalid_element_found` foi introduzido para identificar elementos inválidos na lista. Se um elemento inválido for encontrado, os contadores são zerados e o loop é interrompido, garantindo resultados precisos.

Validação de Input Uma validação foi adicionada para verificar se o valor de `number` é um número inteiro antes da execução do código, prevenindo erros inesperados. Benefícios das Melhorias:

3.7. Conclusão da Experiência

O código apresentado pela ferramenta como resultado tem a saída final desejada, logo consideramos esse teste um sucesso, no entanto ainda há um ponto a se considerar. O ChatGPT adicionou uma condição de parada adicional ao programa que não está explícita na modelagem, consideramos isso um ponto positivo nesse caso, já que para rapidamente a checagem na primeira falha, mas vale cautela em uma implementação real pois esse tipo de alteração pode gerar comportamento anômalo no código. A ferramenta demonstrou diversas características positivas, como facilidade de uso, ser gratuita e elaborar bem as causas dos erros e as soluções propostas, em geral a experiência foi positiva e consideramos interessante realizar mais testes com implementações mais complexas.

4. Conclusão

O uso de LLMs em todos os ciclos da engenharia de software é uma realidade insuperável. Apesar de muitas aplicações ainda estarem incipientes, o seu uso na inspeção de código já é muito vantajoso para o desenvolvedor. É patente que quanto maior a complexidade menor é a qualidade da inspeção, vide o resultado que obtivemos com o nosso teste - de código mais simples - em comparação com os resultados obtidos nos artigos, porém o tempo tem sido um fator muito importante na melhora dessas LLMs, e ele tem sido cada vez mais curto entre um salto de qualidade e outro. Levando a crer que o uso desses auxiliares se tornará cada vez mais indispensável nas inspeções de código, podendo até mesmo se tornar a regra para entregar um produto menos defeituoso, mais barato e de maior facilidade de manutenção.

References

- [1] N. Al Madi, "How Readable is Model-generated Code? Examining Readability and Visual Inspection of GitHub Copilot,".
- [2] F. Feyzi, "Using Large Language Models to Better Detect and Handle Software Vulnerabilities and Cyber Security Threats,".
- [3] Z. Szabó and V. Bilicki, "Introdução ao Uso de Inteligência Artificial na Detecção de Vulnerabilidades de Segurança em Projetos de Software,".
- [4] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, "Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities,".
- [5] H. Li, Y. Hao, Y. Zhai, and Z. Qian, "Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach,".